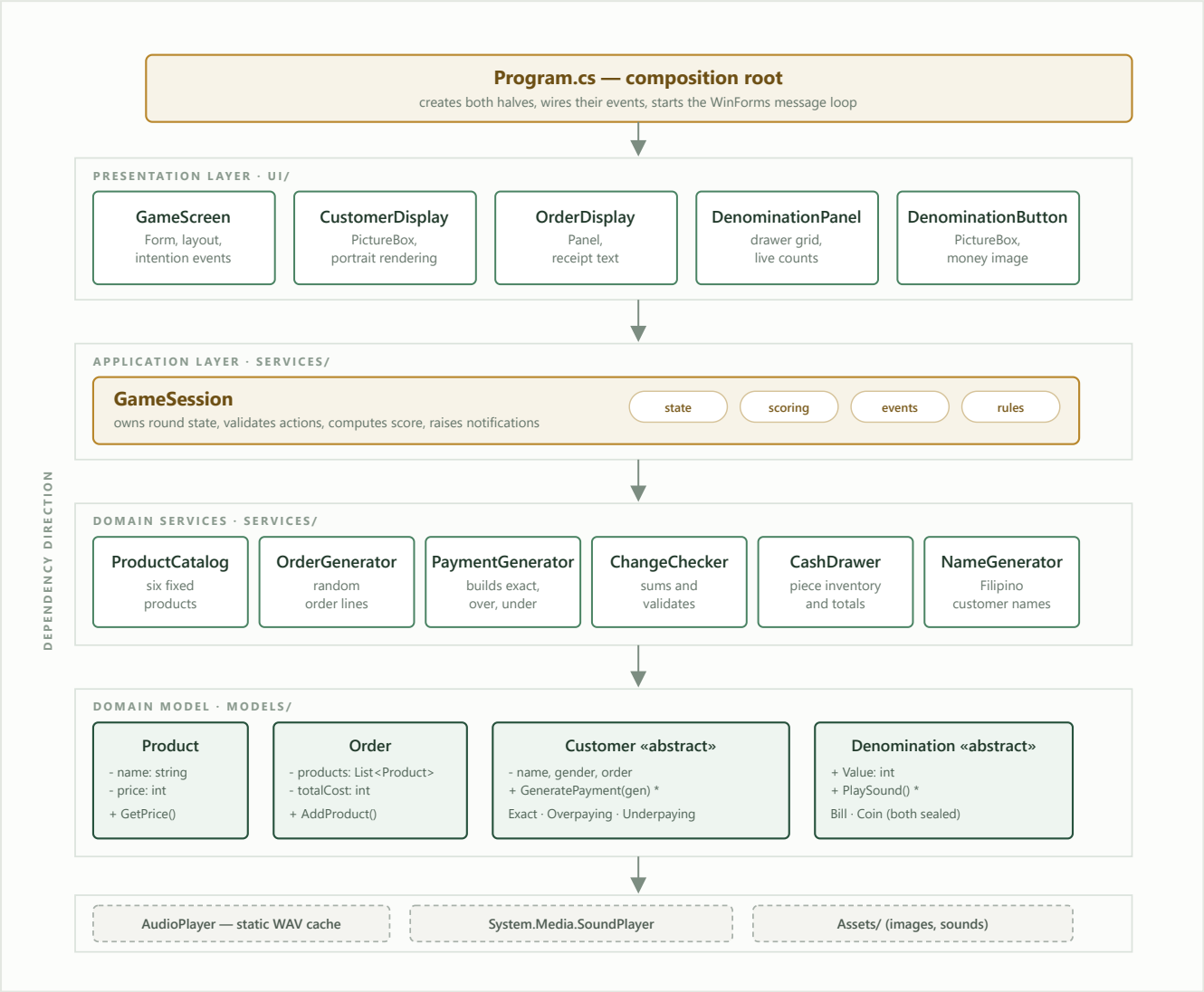
ARCHITECTURE BRIEF

Tindahan ni Aling Nena (Recoded)

System architecture and the four pillars of object-oriented programming



Layered architecture with a passive view. Code dependencies point downward only. The UI raises intention events; `GameSession` validates them and notifies back through `StateChanged`, `MessageSent`, and `CustomerStarted`. The lower layers never reference UI types.

COMPOSITION ROOT

Program.cs constructs the session and the form, then wires the eight subscriptions.

PRESENTATION

WinForms controls that render state and translate clicks into events.

APPLICATION

GameSession holds the round: customer, payment, selection, score, drawer.

DOMAIN

Models plus stateless services: products, orders, money, change rules.

Design intent. The architecture separates what the player sees from the rules being enforced. A round is validated by `ChangeChecker` against the order total and amount paid, while the drawer is a shared inventory of nine reusable denomination objects.

OBJECT-ORIENTED ORGANIZATION

The Four Pillars in This Program

Each pillar mapped to the classes, files, and mechanisms that actually implement it

MODELS/ · MONEY/ · SERVICES/

1. Encapsulation

State is private; the class exposes only the operations that preserve its invariants.

- `Product` keeps `name` and `price` private behind getters.
- `Order.totalCost` changes only in `AddProduct`, so the running total cannot drift.
- `CashDrawer.pieces` is mutated only by `AddPiece`, `AddPieces`, `RemovePiece`.
- `Bill` / `Coin` use private constructors; callers receive shared instances from a static registry.

MODELS/ · SERVICES/

2. Abstraction

Base types state *what* must happen; subclasses and services supply the *how*.

- `Customer` is abstract and declares `GeneratePayment(PaymentGenerator)` with no body.
- `Denomination` is abstract: `Value`, `ImagePath`, and abstract `PlaySound()`.
- `ChangeChecker` hides arithmetic behind `GetTotal`, `CountPieces`, and `IsChangeCorrect`.
- `GameSession` exposes commands and events, not its internal round fields.

MODELS/CUSTOMERS/ · MODELS/MONEY/

3. Inheritance

Two small, closed hierarchies share identity and behaviour through a common base.

- `ExactPayingCustomer`, `OverpayingCustomer`, and `UnderpayingCustomer` derive from `Customer`.
- `Bill` and `Coin` derive from `Denomination` and are marked `sealed`.
- Subclasses reuse the base name, gender, and order fields and override only payment generation / sound.
- UI classes also inherit framework behaviour: `GameScreen : Form`, `OrderDisplay : Panel`.

RUNTIME DISPATCH · EVENTS

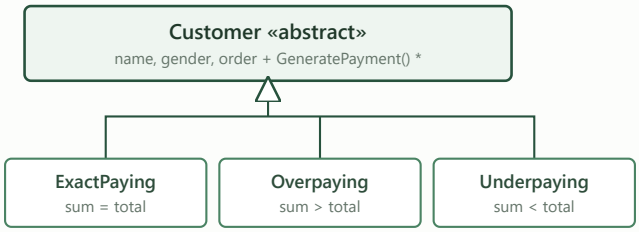
4. Polymorphism

One call site, several runtime behaviours — through virtual methods and delegates.

- `piece.PlaySound()` runs `Bill.PlaySound` or `Coin.PlaySound` from one loop.
- `customer.GeneratePayment(gen)` selects exact, overpaying, or underpaying behaviour at runtime.
- Events (`Action<T>`) let `Program` bind handlers without the publisher knowing the subscriber.
- Control overrides such as `OnPaint` and `Dispose` specialise framework behaviour.

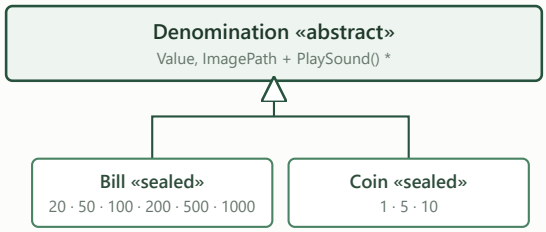
HOW THE HIERARCHIES ARE ORGANISED

PAYER HIERARCHY



Each subclass overrides `GeneratePayment`; the session treats all three as a `Customer`.

MONEY HIERARCHY



Nine shared instances total, created once in static registries.

Organising rule: abstract base for shared state and contract, concrete subclasses for the varying behaviour, `sealed` where the hierarchy ends.

■ Abstract base ☐ Concrete subclass Hollow arrow points to the base class